The `@DataJdbcTest` annotation encapsulates handy features. For example, it makes each test method run in a transaction and rolls it back at its end, keeping the database clean. After running the test method in listing 5.18, the database will not contain the book created in `findBookByIsbnWhenExisting()` because the transaction is rolled back at the end of the method's execution.

Let's verify that the Testcontainers configuration works. First, make sure the Docker Engine is running in your local environment. Then open a Terminal window, navigate to the root folder of your Catalog Service project, and run the following command to ensure the tests are successful. Under the hood, Testcontainers will create a PostgreSQL container before the test's execution and remove it at the end.

```
$ ./gradlew test --tests BookRepositoryJdbcTests
```

In the code repository accompanying the book, you can find more examples of unit and integration tests for the Catalog Service project. The following section will cover how to run full integration tests with Testcontainers.

5.3.3 Integration tests with @SpringBootTest and Testcontainers

In the Catalog Service application, we already have a `CatalogServiceApplicationTests` class annotated with `@SpringBootTest` and containing full integration tests. The Testcontainers configuration we defined earlier applies to all autotests for which the `integration` profile is enabled, so we need to add the profile configuration to the `CatalogServiceApplicationTests` class.